

Reviewer Pack — Minimal Eta-Invariant Correlation with Resolution Proof Width...

Entry 7b586ef6ce8f · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-01 02:07:43 UTC

Minimal Eta-Invariant Correlation with Resolution Proof Width

Entry ID: 7b586ef6ce8f **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-01 02:07:43 UTC

1. Conjecture

Field A (mathematical branch): Arithmetic Geometry (specifically, Eta-Quotients) **Field B** (complexity object): Resolution Proof Complexity

Statement:

For every d -regular graph G , the minimal eta-invariant ($\eta(G)$) of its associated Tseitin formula φ_G is linearly correlated with its resolution proof width $w(\varphi_G)$, such that $\eta(G) = \Theta(w(\varphi_G))$.

Rationale (proposer's reasoning):

Eta-quotients provide a measure in arithmetic geometry that could potentially capture the complexity of computation through their algebraic structure. If this invariant correlates with the resolution proof width, it might expose hidden structures related to computational complexity.

Taxonomy category: Arithmetic Geometry × Resolution Proof Complexity (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): f71210dc1f5dd8db

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For a d -regular graph G , if the correlation coefficient between $\eta(G)$ and $w(\varphi_G)$ is ≥ 0.8 with a p -value ≤ 0.05 over 30 random seeds, or any seed produces $\eta(G) / w(\varphi_G) > 2$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	1.00	SAFE	SAFE
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "eta-invariant" AND "resolution proof width" AND arithmetic geometry" - "Tseitin formula" AND minimal eta-invariant AND graph theory" - "correlation" AND "resolution complexity" AND "arithmetic geometry"

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def gaussian_elimination(A, b):
    n = len(b)
    for i in range(n):
        # Find pivot row
        max_row = i
        for k in range(i + 1, n):
            if abs(A[k][i]) > abs(A[max_row][i]):
                max_row = k

        # Swap rows
        A[i], A[max_row] = A[max_row], A[i]
        b[i], b[max_row] = b[max_row], b[i]

        # Eliminate below pivot
        for k in range(i + 1, n):
            factor = Fraction(A[k][i], A[i][i])
            for j in range(i, n):
                A[k][j] -= factor * A[i][j]
            b[k] -= factor * b[i]

    # Back-substitution
    x = [0] * n
    for i in range(n - 1, -1, -1):
        x[i] = Fraction(b[i], A[i][i])
        for k in range(i - 1, -1, -1):
            b[k] -= A[k][i] * x[i]

    return x
```

```

def eta_quotient(G, u, v):
    n = len(G)
    M = [[0] * (n + 1) for _ in range(n + 1)]
    b = [0] * (n + 1)

    # Fill matrix M and vector b
    for i in range(n):
        for j in range(i + 1, n):
            if G[i][j]:
                M[u][i] += 1
                M[v][j] += 1
                M[i][v] += 1
                M[j][u] += 1

    # Add slack variables
    for i in range(n):
        M[n][i] = -1
        b[n] += 1

    # Solve linear system using Gaussian elimination
    solution = gaussian_elimination(M, b)

    # Compute eta-quotient
    eta = Fraction(0)
    for i in range(n):
        if solution[i] != 0:
            eta += Fraction(1, solution[i])

    return eta

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = random.randint(5, 40)
    d = random.randint(2, min(2 * (n - 1), 40))

    # Generate random d-regular graph
    G = [[0] * n for _ in range(n)]
    edges = []
    for u in range(n):
        for v in range(u + 1, n):
            if len([w for w in range(n) if G[u][w] + G[v][w] == d]) < d:
                G[u][v] = 1
                G[v][u] = 1
                edges.append((u, v))

    # Construct Tseitin formula and compute eta-invariant
    eta_values = [eta_quotient(G, u, v) for u in range(n) for v in range(u + 1, n)]
    eta_mean = sum(eta_values) / len(eta_values)

    # Compute resolution proof width (simplified example)
    w_phi_G = len(edges) * d

    return {
        "metric_name": "eta_invariant",
        "metric_value": eta_mean,
        "instances_tested": n * (n - 1) // 2,
    }

```

```

        "n_max": n,
        "conjecture_holds": False,
        "counterexample": "mapping_undefined"
    }

if __name__ == "__main__":
    import sys
    seeds = [int(arg) for arg in sys.argv[1:]]
    if not seeds:
        seeds = [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71, 73, 79]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_eta = sum(res["metric_value"] for res in results) / len(results)
    std_eta = math.sqrt(sum((res["metric_value"] - mean_eta) ** 2 for res in results) / len(results))
    support_fraction = sum(1 for res in results if res["conjecture_holds"]) / len(results)

    if all(res["conjecture_holds"] for res in results):
        print(f"RESULT: SUPPORTED mean={mean_eta} std={std_eta} support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_eta} std={std_eta} support_fraction={support_fraction}")
    else:
        first_failing_seed = next(res["seed"] for res in results if not res["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample='mapping_undefined' first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_d5f924f0.py", line 116, in <module>
 result = run_trial(seed)

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_d5f924f0.py", line 93, in run_trial
 eta_values = [eta_quotient(G, u, v) for u in range(n) for v in range(u + 1, n)]

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_d5f924f0.py", line 67, in eta_quotient
 solution = gaussian_elimination(M, b)

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_d5f924f0.py", line 33, in gaussian_elimination
 factor = Fraction(A[k][i], A[i][i])

File "/usr/lib/python3.12/fractions.py", line 281, in __new__
 raise ZeroDivisionError('Fraction(%s, 0)' % numerator)

ZeroDivisionError: Fraction(0, 0)

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed during execution due to a division by zero error, which prevented it from producing any data or results. | next: Review the code for potential causes of the division by zero and fix the issue. Once fixed, rerun the test with a larger sample size to ensure robustness.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13721
2	preregistration	ollama_remote	glm4:latest	0	0	15256
3	novelty	ollama_remote	glm4:latest	0	0	10067
4	novelty	ollama_remote	glm4:latest	0	0	9107
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16961
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12636
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	21025
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14270
9	judge	ollama_remote	glm4:latest	0	0	9394

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 122438 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/7b586ef6ce8f.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/7b586ef6ce8f.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/7b586ef6ce8f.tar.gz (if generated)